

Invariants without Targets: Learning Inductive Loop Summaries with Execution-Grounded, Hack-Resistant Rewards

Anonymous Author(s)
Anonymous Institution
anonymous@example.com

July 9, 2026

Abstract

Loop invariant generation is a central challenge in program verification, and four decades of work—abstract interpretation, constraint solving, data-driven learning, and most recently large language models—have framed it almost uniformly as a *target-directed* task: given a program *and a verification target* (a postcondition or assertion), find an inductive invariant strong enough to discharge the target. We argue that this framing rests on a circularity that limits practical impact: verification targets are scarce in real code, writing them is itself the specification bottleneck, and for loop programs the target is typically nothing but the projection of an invariant at loop exit ($I \wedge \neg B \Rightarrow Q$)—whoever wrote Q had already done the invariant reasoning. We therefore reframe the task as generating an *inductive loop summary*: the strongest practically expressible inductive invariant that characterizes a loop’s reachable states, synthesized *closed-book*, with no postcondition visible to the generator. Removing the target removes the binary verifier signal that prior learning approaches optimize, so we develop an execution-grounded dense reward suitable for reinforcement learning: candidate invariants are first reduced to an inductive subset by Houdini-style pruning, then scored by the fraction of *truthful negatives*—trace units that are provably impossible under the loop’s dynamics, constructed along a relational dimension and a bound dimension—that the surviving subset rejects. Because such a reward is a natural target for reward hacking, we harden it with a complexity gate, a per-rollout atom budget, unique-kill junk discounting, and multi-seed minimum scoring against held-out sampling seeds backed by seed-hashed far probes and tier jitter, making memorization and predicate-spraying strictly dominated by genuine invariants. The result, LOOPGYM, is an open three-component pipeline—sampler, reward service, and inference framework—in which the reward passes three adversarial validation gates (discrimination, hack sweep, and mislabel audit) with zero violations over 366 linear and nonlinear benchmark programs, and generated summaries are certified end-to-end by Frama-C/WP.

1 Introduction

Loop invariants are the linchpin of deductive program verification. Ever since the inductive-assertion method of Floyd [Floyd, 1967] and the axiomatic semantics of Hoare [Hoare, 1969], verifying a loop `while (B) S` against a contract $\{P\} \cdot \{Q\}$ has reduced to exhibiting

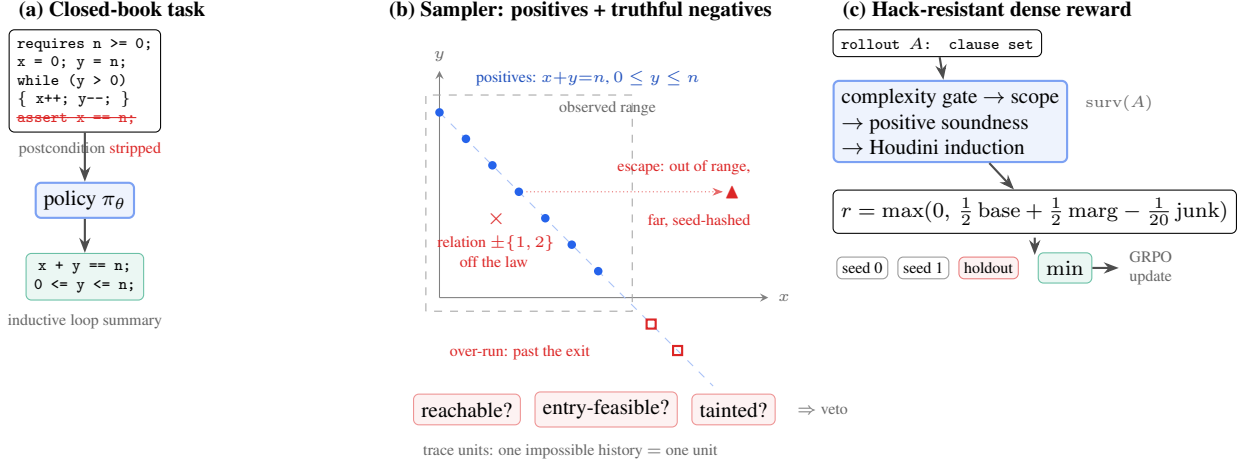


Figure 1: **LOOPGYM overview.** (a) The policy sees only precondition, guard, and body—the postcondition is stripped—and must emit the *strongest* inductive loop summary, not an invariant tailored to a given target. (b) The sampler executes the instrumented loop over a deterministic, precondition-respecting input sweep; positives are observed loop-head states, and negatives are *truthful impossible traces* in two dimensions: relational perturbations off densely sampled trace windows, and bound violations (over-run continuations past a genuine exit, and seed-hashed range escapes). Three vetoes (reachability, entry feasibility, nondeterminism taint) guarantee every emitted negative is unreachable under its input. (c) Each rollout is reduced to its surviving inductive core, scored by the fraction of impossible traces it rejects (plus ablation-based marginal credit and a junk discount), and *minimized* across canonical and held-out sampling seeds, so memorizing any finite sample earns nothing.

an *inductive invariant* I satisfying initiation ($P \Rightarrow I$), consecution ($\{I \wedge B\} S \{I\}$), and safety ($I \wedge \neg B \Rightarrow Q$). Finding I automatically has consequently been one of the most intensively studied problems in formal methods. Four decades of research have attacked it with abstract interpretation [Cousot and Cousot, 1977, Cousot and Halbwachs, 1978, Karr, 1976], constraint- and template-based synthesis [Colón et al., 2003, Sankaranarayanan et al., 2004, Gulwani et al., 2008, Gupta and Rybalchenko, 2009], interpolation and IC3/PDR-style generalization [McMillan, 2003, Bradley, 2011, Komuravelli et al., 2014], abductive inference [Dillig et al., 2013], syntax-guided and stochastic search [Alur et al., 2013, Fedyukovich et al., 2017], data-driven learning from program states [Sharma et al., 2012, 2013, Garg et al., 2014, 2016, Padhi et al., 2016, Zhu et al., 2018, Ezudheen et al., 2018], neural architectures [Si et al., 2018, 2020, Ryan et al., 2020, Yao et al., 2020], and, most recently, large language models [Pei et al., 2023, Kamath et al., 2023, Chakraborty et al., 2023, Wu et al., 2024b,a, Liu et al., 2024b, Wen et al., 2024, Liu et al., 2024a].

The target-dependence problem. Despite their diversity, these lines of work share a common task definition: the input is a program *together with a verification target*—a postcondition, assertion, or safety property Q —and the output is an invariant just strong enough to discharge Q . The target is not incidental; it is load-bearing. CEGIS- and ICE-style learners derive their counterexamples from failed attempts to prove Q [Garg et al., 2014, 2016, Padhi et al., 2016]; Code2Inv shapes its reinforcement signal from the solver’s rejection of Q -directed verification conditions [Si et al., 2018, 2020]; LLM pipelines put the

assertion in the prompt and score candidates by whether the verifier closes the goal [Kamath et al., 2023, Chakraborty et al., 2023, Wu et al., 2024b,a, Liu et al., 2024b].

We contend that this framing rests on an uncomfortable circularity. First, verification targets are *scarce*: real code overwhelmingly ships without formal postconditions, and writing them is itself the specification bottleneck that specification mining and, lately, LLM-based specification generation have tried to relieve [Ernst et al., 2007, Endres et al., 2024, Ma et al., 2025, Wen et al., 2024]. Second, and more fundamentally, for loop programs the verification target is very often nothing more than *the invariant itself evaluated at loop exit*. The safety condition $I \wedge \neg B \Rightarrow Q$ means that the natural Q for a loop—the one a human or a benchmark author writes down—is the exit-time projection of an invariant the author already had in mind; Furia and Meyer make the reverse direction explicit by *deriving* candidate invariants syntactically from postconditions [Furia and Meyer, 2010]. A system that requires Q in order to find I therefore presupposes the very reasoning it is supposed to automate. Target-directed training compounds the problem: when the reward is “did the verifier close Q ,” the learner is incentivized to produce the *weakest* invariant sufficient for the given target—frequently a light rewriting of Q itself—which transfers to no other property of the same loop.

Our reframing: inductive loop summaries. We instead study the task of generating an inductive *loop summary*: given only the precondition, guard, and body—*no postcondition*—produce the strongest practically expressible inductive invariant characterizing the loop’s reachable states at the loop head (Figure 1a). This framing follows the tradition of loop summarization in static analysis and symbolic execution [Kroening et al., 2008, Godefroid and Luchaup, 2011, Xie et al., 2016, Farzan and Kincaid, 2015, Kincaid et al., 2017], but asks for summaries in the standard inductive-invariant form consumed by deductive verifiers (ACSL, checked by Frama-C/WP [Kirchner et al., 2015, Baudin et al., 2021]). A tight summary is strictly more useful than a target-directed invariant: it discharges *every* downstream assertion Q with $I \wedge \neg B \Rightarrow Q$, it is reusable across changing specifications, and generating it exercises exactly the semantic reasoning—conservation laws, relational couplings, progress bounds—that the target-directed setting lets a system shortcut.

The reward problem, and why it is hard. Dropping the target, however, destroys the training signal every prior learning-based system relied on: with no Q there is no binary verified/failed outcome to optimize. “Is I inductive?” alone is useless as a reward—*true*, the guard, and any tautology are inductive. Recent work concurs that the verifier bit is too coarse a signal for learning even in the target-directed setting: InvBench scores invariants by wall-clock speedup rather than by verification alone [Wei et al., 2025], and Pinto et al. [2026] show that grading and curating invariants by quality, not mere validity, is what makes them useful training data. What must be rewarded is *soundness plus tightness*: how much of the unreachable state space does the candidate provably exclude? We operationalize this with an execution-grounded dense reward (Figure 1b,c). A sampler compiles and runs the instrumented loop over a deterministic input sweep to collect *positive* loop-head states, then constructs *truthful negatives*—trace units that are provably impossible under the loop’s own dynamics—along two balanced dimensions: a *relational* dimension (small perturbations off densely observed trace neighborhoods, exposing missing laws such as $x + y = n$) and a *bound* dimension (post-exit continuations of real traces and box-escape states, exposing missing range facts). A candidate rollout is first reduced to its inductive core by Houdini-

style pruning [Flanagan and Leino, 2001], with Frama-C/WP as the authoritative backend, and the surviving subset is scored by the fraction of negative traces it rejects, plus a marginal (ablation) term for group-relative credit assignment.

Such a reward is a textbook target for reward hacking [Amodei et al., 2016, Skalse et al., 2022, Gao et al., 2023]: a policy can memorize the sampled negative pool with sprays of disequalities, or fit sample-extreme bounding boxes, without learning anything semantic. We treat hack resistance as a first-class design requirement and harden the reward with a per-invariant complexity gate, a per-rollout atom budget, unique-kill junk discounting, decade-spread seed-hashed negative placement (defeating both pointwise and interval-window memorizers), and—crucially—multi-seed *minimum* scoring against held-out sampling seeds whose input envelopes are decorrelated by seed-hashed far probes and tier jitter, so that any predicate fitted to one concrete sample collapses on the others while true invariants are seed-indifferent.

Contributions. This paper makes the following contributions:

- **Task reframing.** We identify the target-dependence circularity in loop invariant generation and formulate the alternative task of *closed-book inductive loop summary generation*, which subsumes target-directed generation (§3).
- **An execution-grounded, hack-resistant reward.** We design a dense reward for this task built on truthful negatives—provably impossible traces constructed from the loop’s own dynamics along relational and bound dimensions—with Houdini-pruned survival, trace-unit accounting, and a layered anti-hacking defense: complexity gate, atom budget, junk discounting, holdout-seed minimum scoring, far probes, tier jitter, slice caps, and dimension balancing (§4).
- **An open three-component pipeline.** LOOPGYM decomposes into an example sampler, a containerized HTTP reward service consumable by any RL trainer, and an inference framework with Houdini repair and Frama-C/WP certification (§3).
- **Adversarial validation of the reward.** Three regression gates—a discrimination suite of gold/loose/adversarial invariant families, a gold-free hack sweep, and a mislabel audit over all generated negatives—pass with zero violations on 366 linear and nonlinear benchmark programs (§5).

2 Related Work

Classical invariant generation. Static approaches compute invariants as fixpoints in abstract domains: Karr’s affine-relation analysis [Karr, 1976], abstract interpretation [Cousot and Cousot, 1977], and the polyhedral domain [Cousot and Halbwachs, 1978] discover invariants of a fixed shape without any target, but are limited to the chosen domain’s expressiveness and lose precision under widening. Constraint- and template-based methods fix a syntactic template and solve for coefficients—linear invariants via Farkas-style non-linear constraint solving [Colón et al., 2003], non-linear invariants via Gröbner bases [Sankaranarayanan et al., 2004], and program analysis recast as constraint solving [Gulwani et al., 2008, Gupta and Rybalchenko, 2009]. In verification-condition frameworks, invariants are by-products of proving a property: predicate abstraction [Graf and Saïdi, 1997, Ball and Rajamani, 2002], Craig interpolation [McMillan, 2003], IC3/PDR [Bradley, 2011, Komuravelli et al., 2014, Gurfinkel et al., 2015], and abduction [Dillig et al., 2013] all generalize from the

negation of the target; without a property to refute, these engines have nothing to generalize from. Houdini [Flanagan and Leino, 2001] computes the greatest inductive subset of a candidate pool; LOOPGYM reuses exactly this algorithm as its soundness backstop, but delegates *candidate generation*—the part Houdini leaves to heuristic annotation guessing—to a learned policy. Furia and Meyer generate invariant candidates by syntactically mutating the postcondition [Furia and Meyer, 2010], the clearest evidence for our motivating observation that in practice the target and the invariant are two projections of the same object.

Data-driven and learning-based invariant inference. Daikon pioneered inferring *likely* invariants from execution traces without any specification [Ernst et al., 2007]; DIG extended this to polynomial and array invariants [Nguyen et al., 2012], later with symbolic-state checking [Nguyen et al., 2022, 2017]. These works share our target-free spirit but stop at “likely”: candidates are unsound until confirmed, and confirmation is left to a downstream prover. Guess-and-check frameworks close that loop: invariant inference as classification over reachable/unreachable states [Sharma et al., 2012, 2013], ICE learning with implication counterexamples [Garg et al., 2014], its decision-tree instantiation [Garg et al., 2016], Horn-ICE for contracts [Ezudheen et al., 2018], data-driven CHC solving [Zhu et al., 2018], and PIE’s on-demand feature learning [Padhi et al., 2016]. Crucially, in all of these the *negative* and *implication* examples are produced by a verifier attempting a given property: the teacher is target-directed even when the learner is agnostic. LOOPGYM keeps the classification view—an invariant is scored by how it separates reachable from unreachable states—but manufactures negatives *from the loop’s own dynamics* (impossible continuations and off-manifold perturbations of real traces), so the signal exists before any property is stated.

Neural and LLM-based invariant generation. Code2Inv learns a policy over invariant syntax trees with reinforcement from solver feedback [Si et al., 2018, 2020]; CLN2INV and G-CLN learn invariant coefficients from traces with differentiable logic [Ryan et al., 2020, Yao et al., 2020]. With LLMs, Pei et al. [2023] fine-tune to predict Daikon-style invariants at scale; Loopy prompts GPT-4 for ACSL invariants repaired by Houdini [Kamath et al., 2023]; iRank shows that repeated test-time sampling reliably places a correct invariant *in the pool* and re-ranks candidates to surface it [Chakraborty et al., 2023]; Lemur interleaves LLM proposals with automated reasoners [Wu et al., 2024b]; LaM4Inv couples LLM candidate generation with bounded model checking in a query-filter-reassemble loop [Wu et al., 2024a]; ACInv extracts static structure (including data-structure shape) into prompts and refines candidates with an LLM judge [Liu et al., 2024b]; AutoSpec and SpecGen synthesize full specifications with static-analysis guidance [Wen et al., 2024, Ma et al., 2025]; benchmarks now probe memory-manipulating programs [Liu et al., 2024a] and proof-oriented languages [Chakraborty et al., 2025, First et al., 2023, Misu et al., 2024]. Almost without exception these systems receive the assertion in the prompt and are judged—and, where trained, rewarded—by whether the target verifies. This yields exactly the failure mode we target: sparse, binary, hackable feedback [Skalse et al., 2022, Gao et al., 2023] that favors restating Q over understanding the loop. LOOPGYM differs on both axes: the generator is *closed-book* (the postcondition is stripped from its input), and the reward is a dense, execution-grounded measure of summary tightness rather than a verifier bit.

Invariant quality as a training signal. A recent line of work recognizes that “verifies” is too coarse a label for learning. InvBench evaluates LLM invariant synthesis by *wall-clock speedup* over a state-of-the-art symbolic verifier and finds that untrained LLMs offer little advantage [Wei et al., 2025]. Pinto et al. [2026] make the case that *not all invariants are equal* for training: raw solver-generated invariants are structurally obfuscated and pedagogically poor, and they curate supervised fine-tuning data by grading invariants on non-degeneracy, correctness, usefulness, and compactness—but the grades still presuppose a postcondition to be useful *for*, and supervised imitation of a solver teacher upper-bounds the student by that teacher. Closest to our reward design, SpecRL trains Dafny specification generators with reinforcement learning against *negative tests*—input–output pairs the program can never produce—using the fraction rejected as a completeness signal [Huang et al., 2026]. We share the insight that rejection of impossible behavior is the right dense proxy for specification strength, but differ in three ways: our negatives are *trace units* inside a loop (impossible histories, not I/O pairs), each carries a construction-time unreachability argument specific to its family (density windows, genuine exits, range escapes) rather than relying on sampling alone, and—because a finite negative pool is itself an attack surface—we treat reward hacking as a first-class threat with layered defenses and adversarial gates, an aspect prior quality-signal work leaves open.

Loop summarization. Summarizing loop effects independently of any property has a long history in static analysis: abstract-transformer summaries [Kroening et al., 2008], partial summaries in dynamic test generation [Godefroid and Luchaup, 2011], disjunctive summaries via path dependency [Xie et al., 2016], compositional recurrence analysis [Farzan and Kincaid, 2015, Kincaid et al., 2017], and acceleration [Bardin et al., 2008]. These techniques are precise on the loop classes their theories cover but brittle outside them. We adopt the task framing—characterize the loop, not the goal—but express summaries as ACSL inductive invariants checked by Frama-C/WP [Kirchner et al., 2015, Baudin et al., 2021], and use learning to cover loop shapes (nondeterministic guards, branch-dependent updates, non-linear laws) that closed-form summarization does not.

RL from verifiable rewards and reward hacking. Post-training LLMs against programmatically checkable signals—unit tests for code [Le et al., 2022, Gehring et al., 2024, Chen et al., 2021], final answers for mathematics [Shao et al., 2024, Lambert et al., 2024, DeepSeek-AI, 2025], or process supervision [Lightman et al., 2024]—has proven far more robust than learned preference models [Ouyang et al., 2022, Rafailov et al., 2023], yet even verifiable rewards are gamed when the checker under-specifies the intent [Amodei et al., 2016, Skalse et al., 2022, Gao et al., 2023]. Our setting sharpens this problem: the reward is computed from a *finite sample* of program behavior, so memorizing the sample is always a candidate policy. The layered defenses of §4.2—capacity limits on the policy’s output, uniqueness-based credit, decade-spread seed-hashed negative placement, and minimum-scoring across decorrelated held-out samples—are, to our knowledge, the first systematic anti-hacking treatment for an execution-based formal-methods reward, and we validate them with adversarial gates rather than by assumption.

3 Problem Formulation and System Overview

3.1 From Target-Directed Invariants to Loop Summaries

Fix a loop program $P = (\phi_{\text{pre}}, B, S)$ over integer variables V : a precondition ϕ_{pre} , a guard B , and a body S . A state $\sigma : V \rightarrow \mathbb{Z}$ is a valuation of V (together with the pre-state values v_0 of parameters, so that summaries may relate current to initial values). Let $\mathcal{R}(P)$ denote the set of states reachable *at the loop head* from some initial state satisfying ϕ_{pre} .

Definition 1 (Inductive invariant). *A predicate I over V is an inductive invariant of P if (i) initiation: every loop-head state reached directly from ϕ_{pre} satisfies I ; and (ii) consecution: $\{I \wedge B\} S \{I\}$. Equivalently, $I \supseteq \mathcal{R}(P)$ and I is closed under the loop’s transition relation.*

Definition 2 (Target-directed generation). *Given (P, Q) , produce any inductive I with $I \wedge \neg B \Rightarrow Q$.*

Definition 3 (Inductive loop summary generation (ours)). *Given P alone, produce an inductive I that is as tight as practically expressible: among predicates in the assertion language, I should exclude as much of the unreachable space $\mathcal{R}(P)$ as possible. The ideal object is the strongest inductive invariant, i.e. (the assertion-language best approximation of) $\mathcal{R}(P)$ itself.*

The summary task subsumes the target-directed one: any Q provable from *some* inductive invariant is provable from the strongest one. It is also strictly harder to game. In the target-directed task, Q itself (weakened to pass initiation) is often an acceptable answer; in the summary task there is no Q to parrot, and weak answers—*true*, the guard, loose boxes—are exactly what the reward of §4 is designed to score low.

In practice a summary is a *set* of ACSL clauses `loop invariant  $e_1; \dots; \text{loop invariant } e_k$` ; whose conjunction is the invariant; typical summaries pair one or two relational laws (e.g. $x = y \cdot z$ preserved by the body) with progress bounds (e.g. $0 \leq i \leq n$). Soundness is never entrusted to the learner: every clause set is reduced to its greatest inductive subset by Houdini pruning [Flanagan and Leino, 2001] over Frama-C/WP [Kirchner et al., 2015] before it is scored or reported, so the system’s output is certified by construction.

3.2 System Overview

LOOPGYM decomposes into three components that share a common core (a lightweight C/ACSL program model and a safe ACSL predicate evaluator) but are independently deployable:

1. Example sampler (§4.1)

Compiles and executes the instrumented loop over a deterministic, precondition-respecting input sweep; emits *positive* loop-head states (real trace states) and *truthful negative* trace units (provably impossible histories) along a relational and a bound dimension. The sampler sees only $(\phi_{\text{pre}}, B, S)$ —never the postcondition.

2. Reward framework (§4.2)

Scores a *group* of rollouts (candidate clause sets sampled for the same program, as in group-relative RL [Shao et al., 2024]). Each rollout is reduced to its surviving inductive core, then scored by the fraction of negative traces it rejects (base), its ablation gain over the rest of the group (marginal), and a junk discount; scores are minimized across independent and held-out sampling seeds. Packaged as a slim containerized HTTP service (POST /reward) so any RL trainer can consume it, plus an offline JSONL/Parquet scorer.

3. Inference framework (§4.3)

Prompts the policy *closed-book* (postcondition stripped) for the strongest ACSL invariants; unions n rollouts, deduplicates, Houdini-prunes, re-annotates, and certifies with Frama-C/WP, re-rolling on failure. Supports cloud APIs and in-process batched vLLM for RL evaluation.

A fourth ingredient—three adversarial evaluation gates (discrimination, hack sweep, mislabel audit)—is not part of the serving path but guards the reward’s integrity as a regression suite (§5).

Running example. Consider a benchmark loop (abridged from our linear suite):

```
1 void foo(int n) { // requires n >= 0
2   int x = 0, y = n;
3   while (y > 0) { x = x + 1; y = y - 1; }
4 }
```

The sampler observes loop-head states such as $(x=0, y=5)$, $(x=1, y=4)$, $\dots$ for $n=5$ and many other inputs. Relational negatives perturb dense trace neighborhoods— $(x=2, y=4)$ with $n=5$ breaks $x + y = n$; bound negatives continue past the real exit— $(x=6, y=-1)$ preserves $x + y = n$ yet is impossible. The summary $\{x + y = n, 0 \leq y \leq n\}$ (with $n \geq 0$ from the precondition) rejects every negative trace and is seed-indifferent, scoring ≈ 1 ; the guard restatement $y \geq 0$ rejects only bound negatives on one axis; a sprayed disequality farm rejects canonical negatives but collapses to the minimum on the held-out seed. Any exit-time fact a user later cares about— $x = n, y = 0, x \geq y$ —follows from the summary, though the generator never saw such a target.

4 Method

Notation. For a program $P = (\phi_{\text{pre}}, B, S)$, a *rollout* A is one candidate clause set produced by one policy sample, and a *group* $\mathcal{A} = \{A_1, \dots, A_G\}$ is the set of rollouts sampled for P in one RL step. The sampler emits positives $\mathcal{P}$ (reachable loop-head states, each carrying its run’s input valuation) and negatives $\mathcal{N}$ grouped into *trace units*: each unit is one impossible history under a concrete input, witnessed by one or more states. $\text{surv}(X)$ denotes the subset of clause set X that survives the filter cascade of §4.2, and $\text{rej}(X) \subseteq \mathcal{N}$ the trace units rejected by X — a unit is rejected iff some surviving clause is false at one of its witness states.

4.1 Sampler: Positives and Truthful Negatives

The sampler is deliberately blind to the postcondition: it parses only the precondition, guard, and body. It instruments the loop to print the loop-head valuation on entry, at every iteration, and at exit, compiles with gcc, and executes over a deterministic, precondition-checked input sweep (edge and moderate magnitudes, systematic tier combinations, plus a few large-magnitude probes that widen the input envelope). Loops run to their real exit; runs that hit the iteration cap are flagged and excluded from any range conclusion. The observed loop-head states are the positives.

Negatives are *truthful by construction*: each family carries its own unreachability argument, and is emitted only where that argument holds.

- **Relational perturbations** apply $\pm\{1, 2\}$ single-axis and pairwise steps to trace states whose next few successors were all sampled; since every reachable in-window neighbor

is known, a surviving perturbation is genuinely off the reachable manifold. They expose missing laws such as $x + y = n$.

- **Over-runs** execute the real body past a genuine exit: they preserve every (even non-linear) relational law while leaving the reachable range, and each continuation forms one trace unit.
- **Escapes** move one variable outside its observed range by ladder steps, kept only when they escape; since traces run to the real exit, escapees are unreachable. Together with over-runs they expose missing range facts.

Three vetoes guarantee label correctness — a mislabeled (actually reachable) negative would penalize exactly the true invariants. States observed in any run are never negatives; states that could constitute a fresh loop entry under their input (parameters at pre-values, initialized locals at their initializers, precondition satisfied) are dropped; and variables tainted by nondeterminism (`unknown()` values, control dependence on nondeterministic branches, and their data-flow closure) are never perturbed. A nondeterministic guard disables the over-run and escape families entirely — the loop can always run longer, so no range is sound. The sampler thus degrades soundly: where no argument applies it emits few or no negatives rather than poisoned ones.

4.2 Reward: Two Scores over Prover-Backed Survivors

Each rollout is first reduced to its sound inductive core by a three-stage cascade. A *syntax scrub* runs the Frama-C kernel on the annotated program and drops the exact loop invariant lines it rejects, so one malformed clause cannot void a whole candidate set. A *positive pre-filter* then drops clauses that reference out-of-scope identifiers or are falsified by some sampled positive — a cheap, prover-free approximation that only removes clauses the prover would reject anyway, saving Houdini iterations. Finally *Houdini pruning* [Flanagan and Leino, 2001] over Frama-C/WP [Kirchner et al., 2015] retains the greatest inductive subset. Because survival ends in a real prover, soundness never rests on the sampled states: predicates overfitted to the sample fail initiation or consecution for unconstrained inputs and are removed by proof.

Surviving clauses are scored against the negative trace units:

$$\text{base}(A) = \frac{|\text{rej}(\text{surv}(A))|}{|\mathcal{N}|}, \quad \text{marg}(A) = \frac{|\text{rej}(\text{surv}(U)) \setminus \text{rej}(\text{surv}(U \setminus A))|}{|\mathcal{N}|}, \quad (1)$$

where $U = \bigcup_{A \in \mathcal{A}} A$ is the group union, and the reward is the weighted sum $r(A) = w_b \text{base}(A) + w_m \text{marg}(A)$ (we use $w_b = w_m = 0.5$). Base measures how much of the impossible space the rollout alone excludes; the marginal term is its ablation gain over the rest of the group — group-relative credit in the spirit of GRPO [Shao et al., 2024]. The marginal term also protects test-time diversity: repeated sampling is known to place correct invariants in the candidate pool [Chakraborty et al., 2023, Wu et al., 2024a], while RL fine-tuning tends to sharpen the sampling distribution [Yue et al., 2025]; since marg pays only for what the rest of the group does not already exclude, rollouts restating one another earn nothing, and the group optimum is a set of complementary summaries. Rejection tests are pure Python with C integer semantics (matching the compiled traces), so the only heavyweight cost is the prover; the distinct clause sets a group needs (per-rollout, union, ablations) are memoized and verified in parallel, and each WP run discharges its goals

concurrently. The group additionally receives $\text{batch} = |\text{rej}(\text{surv}(U))|/|\mathcal{N}|$ and a re-roll flag when it falls below a threshold.

4.3 Inference: Closed-Book Generation with Certified Repair

At inference time the policy is prompted with the program after postcondition stripping and asked for the strongest inductive ACSL loop invariants. The framework unions the clauses of n rollouts, deduplicates, and applies the same cascade as the reward — syntax scrub, pre-filter, Houdini — ending in Frama-C/WP verification of the re-annotated program. Repair is Houdini-style subtraction rather than conversational self-correction: unsound clauses are dropped by the pruner, not re-prompted; on failure the framework re-rolls a bounded number of times. Training and inference share one canonical sampler configuration and one filter cascade, so the reward optimized during RL is exactly the quantity the deployed pipeline measures.

5 Experiments

5.1 Setup

Benchmarks. We evaluate on 366 C loop programs in two suites. **Linear** (316 programs) contains single-loop affine programs in the style of SV-COMP loop tracks [Beyer, 2023]; roughly 40% use nondeterministic guards or branches (`unknown()`), exercising the sampler’s soundness rescues. **NLA** (50 programs) contains the standard non-linear arithmetic suite [Nguyen et al., 2012, 2022]: polynomial identities (cubes, geometric sums), gcd/extended-gcd, and division-by-subtraction algorithms whose summaries require products and higher-degree terms (e.g. $y = 3n^2 + 3n + 1$, $ps - rq = 1$, $z = xy$). Every program carries a trailing assertion; it is used *only* for final end-to-end checking and is hidden from the sampler and the generator throughout.

Backends and configuration. Positives/negatives are sampled with 12 runs per seed under the canonical 20-tier sweep; inductiveness and final certification use Frama-C/WP with Z3 (30 s per goal, Typed memory model); reward weights and caps are the defaults of §4.2. The reward runs as a slim container (no prover, no GPU dependencies) and scores a rollout group in the low seconds per program, dominated by Houdini when the prover is enabled.

5.2 Adversarial Validation of the Reward

A reward for RL is only as good as its worst exploit, so before any policy training we validate the reward itself with three adversarial regression gates run across both suites.

Gate 1: discrimination. For each program we score hand-constructed families: *gold* (true tight summaries), *loose* (sound but weak, e.g. guard restatements, slack boxes), and *adversarial* (mega-conjunctions, split sprays, gold-plus-spray, sample-extreme boxes, point-wise farms). The gate requires the strict quality order $\text{gold} \geq \text{loose} > \text{hack pointwise}$, memorizers at least 0.15 below gold, and sample-extreme boxes collapsing on the holdout seed. **Result: 0 violations.**

Gate 2: gold-free hack sweep. Independent of any gold reference, parametric adversary families sweep their capacity (clause counts, atom counts, farm sizes) against every program; the gate requires that no adversary configuration attains the reward of a true summary on any program. **Result: 0 violations.**

Gate 3: mislabel audit. Truthfulness of negatives is audited directly: every emitted negative trace unit is re-checked against exhaustive forward reachability within the audited window, and any reachable state labeled negative is a hard failure (it would penalize true invariants). **Result: 0 hard mislabels** across all 366 programs, including the nondeterministic subset where negatives come only from the rescue constructions.

Together the gates certify the two properties a hack-resistant verifiable reward needs: *soundness* (no true invariant is punished) and *discrimination* (no known exploit outscores truth). They run as a regression suite, so any future change to the sampler or reward must re-pass them.

5.3 Policy Training and End-to-End Generation

Policy-training results are in progress and will be reported in the final version: we train a locally deployable code model with GRPO against the containerized reward service, and evaluate (a) summary tightness on held-out programs, (b) the Frama-C-certified summary rate, and (c) the *downstream* rate at which the closed-book summary alone discharges the benchmark’s hidden assertion—the direct test of the claim that summaries subsume targets. Ablations remove each anti-hacking layer in turn; by construction of the gates in §5.2 we can measure not only reward inflation but the reappearance of specific exploit families.

6 Conclusion

Loop invariant generation has been studied for four decades almost exclusively as a target-directed task, yet the verification target it presupposes is scarce in practice and is, for loops, typically the exit-time shadow of the very invariant being sought. We reframed the problem as *closed-book inductive loop summary generation*—characterize the loop, not the goal—and showed that the missing ingredient for learning this task is not a verifier bit but a dense, execution-grounded, hack-resistant reward. LOOPGYM supplies it: truthful negatives constructed from the loop’s own dynamics along relational and bound dimensions, Houdini-pruned survival for soundness, trace-unit scoring with group-relative marginal credit, and a layered defense (capacity gates, uniqueness discounting, holdout-seed minimum scoring over decorrelated samples) that our adversarial gates certify with zero violations over 366 linear and non-linear programs. The pipeline is deliberately modular—sampler, containerized reward service, certified inference—so that any RL trainer can optimize against it and any deductive toolchain can consume its output.

Beyond the immediate system, we see the two central ideas as portable. Truthful negatives generalize to any domain where a checker can certify impossibility more cheaply than it can certify optimality; minimum-scoring over decorrelated held-out samples is a general recipe for hardening finite-sample verifiable rewards. Future work includes scaling summaries to multi-loop and heap-manipulating programs [Liu et al., 2024a], richer assertion languages (quantifiers, arrays), and closing the loop with the specification-generation literature: a tight loop summary is, after all, the specification.

References

- Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Proc. Formal Methods in Computer-Aided Design (FMCAD)*, pages 1–8, 2013.
- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.
- Thomas Ball and Sriram K. Rajamani. The SLAM project: Debugging system software via static analysis. In *Proc. 29th ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*, pages 1–3, 2002.
- Sébastien Bardin, Alain Finkel, Jérôme Leroux, and Laure Petrucci. FAST: Acceleration from theory to practice. *International Journal on Software Tools for Technology Transfer*, 10(5):401–424, 2008.
- Patrick Baudin, Pascal Cuoq, Jean-Christophe Filliâtre, Claude Marché, Benjamin Monate, Yannick Moy, and Virgile Prevosto. ACSL: ANSI/ISO C specification language. <https://frama-c.com/html/acsl.html>, 2021.
- Dirk Beyer. Competition on software verification (SV-COMP). <https://sv-comp.sosy-lab.org/>, 2023.
- Aaron R. Bradley. SAT-based model checking without unrolling. In *Proc. Int. Conf. on Verification, Model Checking, and Abstract Interpretation (VMCAI)*, pages 70–87. Springer, 2011.
- Saikat Chakraborty, Shuvendu K. Lahiri, Sarah Fakhoury, Madanlal Musuvathi, Akash Lal, Aseem Rastogi, Aditya Senthilnathan, Rahul Sharma, and Nikhil Swamy. Ranking LLM-generated loop invariants for program verification. In *Findings of the Association for Computational Linguistics: EMNLP*, pages 9164–9175, 2023.
- Saikat Chakraborty, Gabriel Ebner, Siddharth Bhat, Sarah Fakhoury, Sakina Fatima, Shuvendu K. Lahiri, and Nikhil Swamy. Towards neural synthesis for SMT-assisted proof-oriented programming. In *Proc. Int. Conf. on Software Engineering (ICSE)*, 2025.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Michael A. Colón, Sriram Sankaranarayanan, and Henny B. Sipma. Linear invariant generation using non-linear constraint solving. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pages 420–432. Springer, 2003.
- Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pages 238–252, 1977.

- Patrick Cousot and Nicolas Halbwachs. Automatic discovery of linear restraints among variables of a program. In *Proc. 5th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pages 84–96, 1978.
- DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Isil Dillig, Thomas Dillig, Boyang Li, and Ken McMillan. Inductive invariant generation via abductive inference. In *Proc. ACM SIGPLAN Int. Conf. on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 443–456, 2013.
- Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K. Lahiri. Can large language models transform natural language intent into formal method postconditions? In *Proc. ACM SIGSOFT Int. Symp. on Foundations of Software Engineering (FSE)*, 2024.
- Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69:35–45, 2007.
- P. Ezudheen, Daniel Neider, Deepak D’Souza, Pranav Garg, and P. Madhusudan. Horn-ICE learning for synthesizing invariants and contracts. In *Proc. ACM SIGPLAN Int. Conf. on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 131:1–131:25, 2018.
- Azadeh Farzan and Zachary Kincaid. Compositional recurrence analysis. In *Proc. Formal Methods in Computer-Aided Design (FMCAD)*, pages 57–64, 2015.
- Grigory Fedyukovich, Samuel J. Kaufman, and Rastislav Bodík. Sampling invariants from frequency distributions. In *Proc. Formal Methods in Computer-Aided Design (FMCAD)*, pages 100–107, 2017.
- Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. Baldur: Whole-proof generation and repair with large language models. In *Proc. 31st ACM Joint European Software Engineering Conf. and Symp. on the Foundations of Software Engineering (ESEC/FSE)*, pages 1229–1241, 2023.
- Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for ESC/Java. In *Proc. Int. Symp. of Formal Methods Europe (FME)*, pages 500–517. Springer, 2001.
- Robert W. Floyd. Assigning meanings to programs. In *Proc. Symp. in Applied Mathematics: Mathematical Aspects of Computer Science*, volume 19, pages 19–32, 1967.
- Carlo Alberto Furia and Bertrand Meyer. Inferring loop invariants using postconditions. In *Fields of Logic and Computation: Essays Dedicated to Yuri Gurevich*, pages 277–300. Springer, 2010.
- Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *Proc. Int. Conf. on Machine Learning (ICML)*, pages 10835–10866, 2023.
- Pranav Garg, Christof Löding, P. Madhusudan, and Daniel Neider. ICE: A robust framework for learning invariants. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pages 69–87. Springer, 2014.

- Pranav Garg, Daniel Neider, P. Madhusudan, and Dan Roth. Learning invariants using decision trees and implication counterexamples. In *Proc. 43rd ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*, pages 499–512, 2016.
- Jonas Gehring, Kunhao Zheng, Jade Copet, Vegard Mella, Taco Cohen, and Gabriel Synnaeve. RLEF: Grounding code LLMs in execution feedback with reinforcement learning. *arXiv preprint arXiv:2410.02089*, 2024.
- Patrice Godefroid and Daniel Luchaup. Automatic partial loop summarization in dynamic test generation. In *Proc. Int. Symp. on Software Testing and Analysis (ISSTA)*, pages 23–33, 2011.
- Susanne Graf and Hassen Saïdi. Construction of abstract state graphs with PVS. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pages 72–83. Springer, 1997.
- Sumit Gulwani, Saurabh Srivastava, and Ramarathnam Venkatesan. Program analysis as constraint solving. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pages 281–292, 2008.
- Ashutosh Gupta and Andrey Rybalchenko. InvGen: An efficient invariant generator. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pages 634–640. Springer, 2009.
- Arie Gurfinkel, Temesghen Kahsai, Anvesh Komuravelli, and Jorge A. Navas. The SeaHorn verification framework. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pages 343–361. Springer, 2015.
- C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- Zhechong Huang, Zhao Zhang, Zeyu Sun, Huifeng Sun, and Yingfei Xiong. Reinforcement learning with negative tests as completeness signal for formal specification synthesis. *arXiv preprint arXiv:2604.05820*, 2026.
- Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models. *arXiv preprint arXiv:2311.07948*, 2023.
- Michael Karr. Affine relationships among variables of a program. *Acta Informatica*, 6(2): 133–151, 1976.
- Zachary Kincaid, Jason Breck, Ashkan Forouhi Boroujeni, and Thomas Reps. Compositional recurrence analysis revisited. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pages 248–262, 2017.
- Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. Frama-c: A software analysis perspective. *Formal Aspects of Computing*, 27(3):573–609, 2015.
- Anvesh Komuravelli, Arie Gurfinkel, and Sagar Chaki. SMT-based model checking for recursive programs. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pages 17–34. Springer, 2014.

- Daniel Kroening, Natasha Sharygina, Stefano Tonetta, Aliaksei Tsitovich, and Christoph M. Wintersteiger. Loop summarization using abstract transformers. In *Proc. Int. Symp. on Automated Technology for Verification and Analysis (ATVA)*, pages 111–125. Springer, 2008.
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, et al. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.
- Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024.
- Chang Liu, Xiwei Wu, Yuan Feng, Qinxiang Cao, and Junchi Yan. Towards general loop invariant generation: A benchmark of programs with memory manipulation. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2024a.
- Ruibang Liu, Minyu Chen, Ling-I Wu, Jingyu Ke, and Guoqiang Li. Enhancing automated loop invariant generation for complex programs with large language models. *arXiv preprint arXiv:2412.10483*, 2024b.
- Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. SpecGen: Automated generation of formal program specifications via large language models. In *Proc. Int. Conf. on Software Engineering (ICSE)*, 2025.
- Kenneth L. McMillan. Interpolation and SAT-based model checking. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pages 1–13. Springer, 2003.
- Md Rakib Hossain Misu, Cristina V. Lopes, Iris Ma, and James Noble. Towards AI-assisted synthesis of verified Dafny methods. In *Proc. ACM SIGSOFT Int. Symp. on Foundations of Software Engineering (FSE)*, 2024.
- ThanhVu Nguyen, Deepak Kapur, Westley Weimer, and Stephanie Forrest. Using dynamic analysis to discover polynomial and array invariants. In *Proc. 34th Int. Conf. on Software Engineering (ICSE)*, pages 683–693, 2012.
- ThanhVu Nguyen, Timos Antonopoulos, Andrew Ruef, and Michael Hicks. Counterexample-guided approach to finding numerical invariants. In *Proc. 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, pages 605–615, 2017.
- ThanhVu Nguyen, KimHao Nguyen, and Matthew B. Dwyer. Using symbolic states to infer numerical invariants. *IEEE Transactions on Software Engineering*, 48(10):3877–3899, 2022.
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

- Saswat Padhi, Rahul Sharma, and Todd Millstein. Data-driven precondition inference with learned features. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pages 42–56, 2016.
- Kexin Pei, David Bieber, Kensen Shi, Charles Sutton, and Pengcheng Yin. Can large language models reason about program invariants? In *Proc. Int. Conf. on Machine Learning (ICML)*, 2023.
- Ido Pinto, Yizhak Yisrael Elboher, Haoze Wu, Nina Narodytska, and Guy Katz. Not all invariants are equal: Curating training data to accelerate program verification with SLMs. *arXiv preprint arXiv:2603.15510*, 2026.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Gabriel Ryan, Justin Wong, Jianan Yao, Ronghui Gu, and Suman Jana. CLN2INV: Learning loop invariants with continuous logic networks. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2020.
- Sriram Sankaranarayanan, Henny B. Sipma, and Zohar Manna. Non-linear loop invariant generation using Gröbner bases. In *Proc. 31st ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*, pages 318–329, 2004.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Rahul Sharma, Aditya V. Nori, and Alex Aiken. Interpolants as classifiers. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pages 71–87. Springer, 2012.
- Rahul Sharma, Saurabh Gupta, Bharath Hariharan, Alex Aiken, Percy Liang, and Aditya V. Nori. A data driven approach for algebraic loop invariants. In *Proc. European Symp. on Programming (ESOP)*, pages 574–592. Springer, 2013.
- Xujie Si, Hanjun Dai, Mukund Raghothaman, Mayur Naik, and Le Song. Learning loop invariants for program verification. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, pages 7751–7762, 2018.
- Xujie Si, Aaditya Naik, Hanjun Dai, Mayur Naik, and Le Song. Code2Inv: A deep learning framework for program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pages 151–164. Springer, 2020.
- Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward gaming. In *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Anjiang Wei, Tarun Suresh, Tianran Sun, Haoze Wu, Ke Wang, and Alex Aiken. InvBench: Can LLMs accelerate program verification with invariant synthesis? *arXiv preprint arXiv:2509.21629*, 2025.

- Cheng Wen, Jialun Cao, Jie Su, Zhiwu Xu, Shengchao Qin, Mengda He, Haokun Li, Shing-Chi Cheung, and Cong Tian. Enchanting program specification synthesis by large language models using static analysis and program verification. In *Proc. Int. Conf. on Computer Aided Verification (CAV)*, pages 302–328. Springer, 2024.
- Guangyuan Wu, Weining Cao, Yuan Yao, Hengfeng Wei, Taolue Chen, and Xiaoxing Ma. LLM meets bounded model checking: Neuro-symbolic loop invariant inference. In *Proc. IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, 2024a.
- Haoze Wu, Clark Barrett, and Nina Narodytska. Lemur: Integrating large language models in automated program verification. In *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024b.
- Xiaofei Xie, Bihuan Chen, Yang Liu, Wei Le, and Xiaohong Li. Proteus: Computing disjunctive loop summary via path dependency analysis. In *Proc. ACM SIGSOFT Int. Symp. on Foundations of Software Engineering (FSE)*, pages 61–72, 2016.
- Jianan Yao, Gabriel Ryan, Justin Wong, Suman Jana, and Ronghui Gu. Learning nonlinear loop invariants with gated continuous logic networks. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pages 106–120, 2020.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? *arXiv preprint arXiv:2504.13837*, 2025.
- He Zhu, Stephen Magill, and Suresh Jagannathan. A data-driven CHC solver. In *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pages 707–721, 2018.

A Sampler and Reward Constants

Table 1 lists the canonical configuration shared by training (reward service) and inference, so that the reward optimized during RL is exactly the quantity measured at deployment.

Constant	Value	Role
runs per seed	12	concrete executions per example set
input tiers	20 tiers, clamped to $[-64, 64]$	deterministic magnitude schedule (edges first)
far probes	3 per seed, magnitude ≤ 512	seed-hashed envelope decorrelation; cube-safe
tier jitter	± 2 on non-edge tiers	seed-hashed value decorrelation
relation deltas	$\pm\{1, 2, 3\}$	small perturbations off dense bases
dense window	4	successors required for a base to be “dense”
escape ladder	$\pm\{5, 8, 13, 21, 34, 55, 89\}$	box-escape bound negatives
over-run steps	24	post-exit continuation length (one trace unit per run)
iteration cap	2×10^6	divergence safety net (capped runs flagged)
slice cap	6	max units per farm-targetable coordinate value
relation:bound	$\leq 2 : 1$	dimension balancing
complexity gate	160 chars / 12 atoms	per-clause capacity limit (drop, not truncate)
rollout atom budget	32	total atoms scored per rollout (prefix)
(w_b, w_m)	(0.5, 0.5)	base / marginal weights (§4.2)
seeds / holdout	2 / 1	minimum-combined example sets
re-roll threshold	0.6	group batch score below which re-rolling is advised
positive subsample	4096	per-set cap; per-variable extremes always retained
Houdini iterations	≤ 500 , patience 2	greatest-inductive-subset pruning
WP configuration	Z3, 30 s, Typed model	per-goal prover budget

Table 1: Canonical constants of the sampler (§4.1), reward (§4.2–§4.2), and verification backend.

B Generation Prompt

The closed-book generation prompt (abridged); the postcondition is stripped from the program before it is shown.

System prompt (abridged)

```
You are given a C function containing a loop. Produce the strongest
inductive ACSL loop invariants that capture the loop's behavior:
- one core conservation / relational equality when one exists;
- minimal tight progress bounds (weaken strict guards to non-strict
  at exit);
- an exact loop assigns clause;
- no tautologies, no restatement of the guard alone;
- reference initial parameter values only via \at(v, Pre);
- allowed operators: == != < <= > >= && || ! ==> <==> + - * / %.
Output ONLY lines of the form:
  loop invariant <expr>;
```

C Reward Service Interface

The reward framework is packaged as a slim container (no prover, no GPU dependencies) exposing POST `/reward` with body `{program, rollouts[]}` and response `{rollout_rewards[], base[], marginal[], rollouts[] (with rejection detail), batch_score, should_reroll}`. Rollouts may be raw LLM text, invariant lists, or annotated code; parsing is uniform. An offline scorer provides the same computation over JSONL/Parquet rollout dumps for trainer-side batching.